COLLEGE DATA SCIENCE MICRO-PROJECT · CRISP-DM

Stock Market Analysis & Price Prediction

Python 3.12 · pandas · scikit-learn · Streamlit `8-page dashboard`
Live yfinance data + offline Kaggle snapshot (5,865 stocks)

01

Objective & CRISP-DM map

Analyze historical stock data collected from an API, then build a full data-science workflow around it — cleaning, feature engineering, exploratory analysis, machine learning, evaluation, prediction, and a live dashboard — all organized around the **CRISP-DM** lifecycle so every stage has a clear purpose and a clear deliverable.

CRISP-DM STAGE	WHAT WE DID	MODULE
Business Understanding	Define the goal: predict next-day/future closing price from historical OHLCV + technical indicators	This document / README
Data Understanding	Pull real daily OHLCV data for any ticker; inspect shape, range, gaps	`data_collection.py`, `bulk_analysis.py`
Data Preparation	Clean (missing values, duplicates, dtypes, outliers) then engineer features (returns, MAs, RSI, MACD, Bollinger, lags)	`preprocessing.py`, `feature_engineering.py`
Modeling	Train and tune 5 regressors, auto-select the best by R^2	`train_model.py`
Evaluation	MAE / MSE / RMSE / R^2 comparison table + 3 diagnostic plots	`evaluate.py`
Deployment	Forecast future prices; serve everything through an interactive dashboard	`predict.py`, `app.py`

02

Tech stack & project structure

DATA

pandas · numpy

Cleaning, feature math, resampling

DATA SOURCE

yfinance

Live Yahoo Finance OHLCV for any ticker

OFFLINE SOURCE

Kaggle dataset

5,865 stock + 2,147 ETF static histories

VISUALIZATION

plotly · matplotlib

15 interactive + static charts

ML

scikit-learn

5 regressors, GridSearchCV, TimeSeriesSplit

PERSISTENCE

joblib

Best model saved per ticker

DASHBOARD

streamlit

8-page interactive app

LANGUAGE

Python 3.12

Type-hinted, logged, modular `src/`

Pipeline modules, in execution order

`config.py` → `data_collection.py` → `preprocessing.py` → `feature_engineering.py` → `eda.py + visualization.py` → `train_model.py` → `evaluate.py` → `predict.py`

`main.py` chains all of these for one ticker from the command line; `app.py` exposes the same logic interactively.

03

Data collection

Two interchangeable sources, both producing the identical schema (`Date, Open, High, Low, Close, Adj Close, Volume`):

1 · Live — yfinance

Any ticker Yahoo Finance recognizes: `AAPL` , `MSFT` , `TSLA` , or international ones like `RELIANCE.NS` , `TCS.NS` . We fetch 1–10 years ending today.

```
python -m src.data_collection --ticker AAPL --years 5
```

2 · Offline — Kaggle bulk snapshot

Kaggle's *jacksoncrow/stock-market-dataset*: 5,865 stock + 2,147 ETF CSVs, frozen at **2020-04-01**, some going back to 1962. Used when offline, for reproducible grading, or for cross-sectional analysis no single live ticker can show.

```
python -m src.data_collection --ticker NFLX --years 5 --source offline
```

A third mode, `--source auto` , tries yfinance first and silently falls back to the offline copy if the network call fails — handy for a live demo with a flaky connection.

04

Cleaning & preprocessing

1. **Missing values** — forward-fill then back-fill. Correct for prices: carry the last known value forward; back-fill only catches a gap at the very start of the series.
2. **Duplicates** — dropped by `Date` , keeping the first occurrence.
3. **Dtypes** — `Date` → datetime, OHLCV → numeric, coercing anything unparseable to `NaN` .
4. **Chronological sort** — required before any rolling-window feature or train/test split makes sense.

5. **Outlier flagging** — IQR method on `Close` (1.5× IQR beyond Q1/Q3). We *flag*, not drop: a market crash is a real data point that belongs in the analysis, not noise to discard.

05

Feature engineering

Every model input beyond raw OHLCV is derived here — 20 feature columns in total.

FEATURE	WHAT IT CAPTURES	FORMULA / LOGIC
Daily Return	Day-over-day momentum	`pct_change(Close) × 100`
MA_20 / MA_50 / MA_100	Short/medium/long trend	Rolling mean of Close
Volatility	Recent riskiness	20-day rolling std of Daily Return
Price_Diff	Absolute day-over-day move	`Close.diff()`
HL_Spread	Intraday range	`High − Low`
RSI (14)	Overbought/oversold momentum	100 − 100/(1 + avg gain ÷ avg loss); neutral=50 on flat windows
MACD / Signal / Hist	Trend-following momentum	EMA(12) − EMA(26); signal = EMA(9) of MACD; hist = MACD − signal
BB_Upper / BB_Lower	Volatility bands around price	20-day MA ± 2 × rolling std
Close_Lag_1/2/3	Autoregressive memory	Close shifted 1/2/3 days

> **WHY THIS MATTERS IN THE VIVA**
>
> RSI, MACD, and Bollinger Bands are standard technical-analysis indicators — including them (rather than just raw price) is what turns this from "curve-fitting yesterday's price" into a feature set closer to what real quant models use.

06

EDA & visualization — 15 charts

Generated per ticker into `reports/figures/` : Plotly for anything worth zooming into interactively, Matplotlib for static distributional plots.

Closing price trend

Long-run price line

Trading volume

Daily volume bars

Candlestick

Full OHLC price action

Moving average comparison

Close vs MA20/50/100

Bollinger Bands

Price vs volatility envelope

RSI

With 30/70 reference lines

MACD

MACD, signal, histogram

Daily returns

Line of % returns over time

Returns histogram

Distribution shape

Returns box plot

Spread & outliers

Correlation heatmap

Feature interdependence

Monthly avg closing price

Seasonality view

Top gain / loss days

Best & worst single days

Rolling volatility

Volatility clustering over time

Autocorrelation

Returns vs 40 lags

> TALKING POINT
>
> Near-zero autocorrelation across lags is the textbook signature of the (weak-form) efficient-market hypothesis — a nice, real statistical result to point to, not just a chart.

07

Machine learning — 5 models, tuned

Target: `Close`. Split: 80/20, **chronological, no shuffling** — the model is always tested on days it has never seen, the only valid protocol for time series.

MODEL	TUNED?	TUNING GRID
Linear Regression	—	No hyperparameters to tune
Decision Tree Regressor	Yes	`max_depth` ∈ {3, 5, 10, None}
Random Forest Regressor	Yes	`n_estimators` ∈ {100, 200}, `max_depth` ∈ {5, 10, None}
Gradient Boosting Regressor	Yes	`n_estimators`, `learning_rate`, `max_depth`
SVR (linear kernel)	—	Target-scaled via `TransformedTargetRegressor`

Tuning uses `GridSearchCV` over a `TimeSeriesSplit` cross-validator, so even the inner validation folds never peek at the future. The best of all 5 models by test-set R^2 is saved with `joblib`.

> THE SVR BUG WE FOUND AND FIXED
>
> An RBF-kernel SVR can't extrapolate past the price range it trained on — on a trending stock, tested on the highest prices it's ever seen, it scored a nonsensical `R² −2.61 ▼`. Switching to a **linear** kernel (which can extrapolate) turned it into the best model on AAPL: `R² 0.9967 ▲`. Same

> story motivates why tree models often trail linear ones here — they literally cannot predict a price higher than any they saw in training.

08

Evaluation

Four metrics computed for every candidate model, ranked by R^2:

METRIC	WHAT IT TELLS YOU
MAE	Average absolute $ error — easiest to explain out loud
MSE	Squared error — penalizes big misses harder
RMSE	Same units as price, so it's directly comparable to MAE
R^2	Fraction of price variance explained — our model-selection metric

Three diagnostic plots per ticker

- **Actual vs Predicted** — points hugging the diagonal = good fit
- **Residual plot** — should look like random noise around zero, not a pattern
- **Prediction error over time** — actual vs predicted lines across the test period

---

09

Prediction

Single-day prediction just feeds the model the most recent row's features. Multi-day forecasting is harder — the model needs tomorrow's Open/High/Low/Volume, which don't exist yet.

Our fix: an **iterative simulation**. Each predicted Close becomes the next day's assumed Open (a persistence assumption), High/Low are approximated as ±1% of that, Volume holds at its recent average, and every technical indicator (MA,

RSI, MACD, Bollinger, lags) is recomputed on the extended series before the next step.

> **BE UPFRONT ABOUT THIS IN THE VIVA**
>
> This is explicitly an educational approximation, not a production forecasting method — say so before anyone asks. It's still the right way to demonstrate multi-step forecasting without inventing data we don't have.

10

Streamlit dashboard — 8 pages

PAGE 1

Dashboard

Metric cards, price trend, volume, MA overlay, CSV export

PAGE 2

Stock Analysis

Date-range-filtered interactive OHLCV charts

PAGE 3

Technical Indicators

Candlestick, Bollinger, RSI, MACD

PAGE 4

EDA

Stats, distributions, correlation, volatility, autocorrelation

PAGE 5

Prediction

Forecast slider + button, history-vs-forecast chart

PAGE 6

Model Performance

Metrics table, tuned hyperparameters, diagnostic plots

PAGE 7

Compare Tickers

Multi-ticker normalized overlay + summary table

PAGE 8

Market Overview

Cross-sectional view of the entire offline dataset

Sidebar controls apply everywhere: ticker input, years of history, a **Data Source** selector (yfinance / offline / auto), and a date-range filter. Every expensive step (`load_pipeline_data` , `load_trained_model`) is `@st.cache` -wrapped so switching pages doesn't re-download or re-train.

11

Offline bulk dataset — cross-sectional analysis

Kaggle's snapshot lets us ask questions no single live ticker can answer: what does the *whole market's* return distribution look like?

Because summarizing ~8,000 files live would make the dashboard unusable, statistics are computed once by a batch script and cached to CSV:

```
python -m src.bulk_analysis --kind stocks --build
python -m src.bulk_analysis --kind etfs --build
```

> A REAL DATA-QUALITY BUG WE CAUGHT
>
> Some legacy tickers (e.g. **NVO**) have corrupted `Adj Close` values from 40+ years of split back-adjustment — prices that underflow to ~1e-9, even briefly negative. We filter these with a minimum valid price before computing any statistic.

> A STATISTICS BUG WE CAUGHT (AND THE FIX THAT MATTERED MORE)
>
> Even with clean prices, raw *total return* isn't comparable across tickers whose histories span 5 years vs 58 years — one blue chip trivially "wins" on compounding alone. We switched the ranking metric to **CAGR** (annualized return), the statistically correct normalization, not just a display clip.

12

Results we actually got

Real numbers from runs during development — good material for slides:

TICKER	SOURCE	BEST MODEL	R^2
AAPL	yfinance	SVR (linear)	0.9967
MSFT	yfinance	SVR (linear)	0.9942
GOOG	yfinance	SVR (linear)	0.9986
NFLX	offline (Kaggle)	Gradient Boosting	0.9924
RELIANCE.NS	yfinance	Linear Regression	0.9917

Compare Tickers page, same period: AAPL +133.9% ▲ GOOG +173.7% ▲ MSFT +44.8% ▲ — all best-modeled by the same linear SVR, which is itself a small finding worth saying out loud.

13

Anticipated viva questions

Q – Why not just use the highest-R^2 model always?

Q – Why does Linear Regression / linear SVR beat Random Forest so often?

Q – Is this actually predicting the future, or just today's price?

Q – What would you add with more time?

Q – Why two data sources?

14

How to run it, live, in class

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

# full CLI pipeline for one ticker
python main.py --ticker AAPL --years 5 --forecast-days 5

# the dashboard
streamlit run app.py
```

For a from-scratch demo, pick a ticker nobody's seen yet — the whole pipeline (download, clean, engineer features, train 5 models, evaluate, forecast) runs in well under a minute.
